

First-Come, First-Served Scheduling

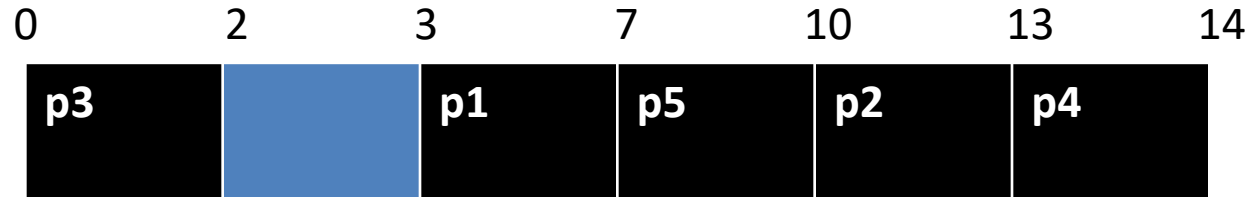
The process which arrives first in the ready queue is firstly assigned the CPU.

In case of a tie, process with smaller process id is executed first.

It is always non-preemptive in nature.

First-Come, First-Served Scheduling

Process Id	Arrival time	Burst time
P1	3	4
P2	5	3
P3	0	2
P4	5	1
P5	4	3



Turn Around time = Exit time – Arrival time
Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
P1	7	$7 - 3 = 4$	$4 - 4 = 0$
P2	13	$13 - 5 = 8$	$8 - 3 = 5$
P3	2	$2 - 0 = 2$	$2 - 2 = 0$
P4	14	$14 - 5 = 9$	$9 - 1 = 8$
P5	10	$10 - 4 = 6$	$6 - 3 = 3$

Advantages

It is simple and easy to understand.

It can be easily implemented using queue data structure.

It does not lead to starvation.

Disadvantages:

It does not consider the priority or burst time of the processes.

It suffers from convoy effect

Shortest Job First:

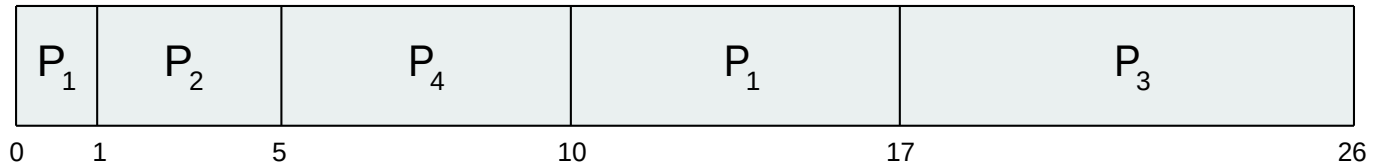
- SJF Scheduling can be used in both preemptive and non-preemptive mode.
- Preemptive mode of Shortest Job First is called as Shortest Remaining Time First (SRTF).
- **for n processes, time complexity = $n \log(n)$**
- **Critaria: Burst Time**

Example of Shortest-remaining-time-first

- Now we add the concepts of varying arrival times and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- Preemptive SJF* Gantt Chart



- Average waiting time = $[(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5$ msec

Shortest Job First:

Preemptive mode of Shortest Job First is called as Shortest Remaining Time First (SRTF); Mood : 1

Process Id	Arrival time	Burst time
p1	3	1
p2	1	4
p3	4	2
p4	0	6
p5	2	3

0 1 2 3 4 5 6 7 8 9 10 11 12 13 16

p4	p2	p2	p1	p2	p2	p3	p3	p5	p5	p5	p4	p4	p4
----	----	----	----	----	----	----	----	----	----	----	----	----	----

0 1 3 4 6 8 11 16

p4	p2	p1	p2	p3	p5	p4
----	----	----	----	----	----	----

Shortest Job First:

Advantages

- ✓ SRTF is optimal and guarantees the minimum average waiting time.
- ✓ It provides a standard for other algorithms since no other algorithm performs better than it.

Disadvantages

- It can not be implemented practically since burst time of the processes can not be known in advance.
- It leads to starvation for processes with larger burst time.
- Priorities can not be set for the processes.
- Processes with larger burst time have poor response time.

Round Robin Scheduling:

- CPU is assigned to the process on the basis of FCFS for a fixed amount of time.
- This fixed amount of time is called as time quantum or time slice.
- After the time quantum expires, the running process is preempted and sent to the ready queue.
- Then, the processor is assigned to the next arrived process.
- higher value of time quantum is better in terms of number of context switch
- smaller value of time quantum is better in terms of response time

Round-Robin Scheduling:

Process Id	Arrival time	Burst time
p1	0	4
p2	1	5
p3	2	2
p4	3	1
p5	4	6
p6	6	3

time quantum = 2 unit

Ready Queue:

P1, P2, P3, P1, P4, P5, P2, P6, P5, P2, P6, P5

0	2	4	6	8	9	11	13	15	17	18	19	21
p1	p2	p3	p1	p4	p5	p2	p6	p5	p2	p6	p5	

Round Robin Scheduling:

Advantages

It gives the best performance in terms of **average response time**.

It is best suited for time sharing system, client server architecture and interactive system.

Disadvantages

It leads to starvation for processes with larger burst time as they have to repeat the cycle many times.

Its performance heavily depends on time quantum.

Priorities can not be set for the processes.